

Ψ Notation: A Formal Framework for Asymptotic Ordering in Multi-Layer Systems

Natarizki

July 8, 2026

Abstract

Classical asymptotic notations such as Big-O, Big-Omega, and Big-Theta are indispensable tools for analyzing the complexity of individual algorithms. However, modern computational systems—ranging from AI inference pipelines to microservice architectures—are inherently multi-layered. In such systems, the *relative* asymptotic behavior between consecutive layers is often more critical than the absolute complexity of any single layer. If an upstream layer grows faster than a downstream layer, the entire pipeline stalls due to resource contention.

This paper introduces **Ψ Notation**, a novel mathematical framework designed to capture and guarantee the asymptotic order between two or three functions residing in the same complexity class $\Theta(g(n))$. We define two variants:

- **Little- ψ** for *two-layer* systems, ensuring $F_1(n) \leq F_2(n)$ asymptotically with a provable constant gap.
- **Big- Ψ** for *three-layer* systems, providing a transitive, unified guarantee for $F_1(n) \leq F_2(n) \leq F_3(n)$.

We present formal definitions, prove key theorems including pairwise implication and strong transitivity, and derive quantitative bounds on the safety margins between layers. Furthermore, we present comprehensive examples covering the five fundamental complexity classes: constant (1), logarithmic ($\log n$), linear (n), linearithmic ($n \log n$), and quadratic (n^2). Both valid and invalid configurations are explicitly demonstrated. A reference Python implementation for automatic validation and constant inference is also provided. This work bridges the gap between pure complexity theory and practical performance engineering for distributed and pipelined architectures.

1 Introduction

The seminal work of Knuth [1] established Big-O, Big-Omega, and Big-Theta as the standard language for describing algorithmic time and space complexity. These notations allow researchers and engineers to classify algorithms based on their asymptotic growth rates, abstracting away constant factors and lower-order terms.

Despite their utility, classical notations are inherently *univariate*. They describe the growth of a single function $f(n)$ relative to a reference function $g(n)$. They provide no

machinery to express relationships between *multiple* functions, particularly the ordering constraint that one function must not asymptotically exceed another.

Consider a modern deep learning inference pipeline. It typically consists of:

1. **Preprocessing** (F_1): Data loading, resizing, normalization, and augmentation.
2. **Inference** (F_2): Matrix multiplications, activation functions, and tensor operations on a GPU.
3. **Postprocessing** (F_3): Softmax, decoding, and serialization to JSON or protobuf.

Even if $F_1 \in O(n^2)$ and $F_2 \in \Theta(n^2)$, it is possible that the hidden constants cause $F_1(n) > F_2(n)$ for all practical n . This creates a CPU bottleneck, starving the GPU and wasting expensive compute resources. Classical notation fails to capture this violation.

Similarly, in microservices, if an authentication service (F_1) scales with $O(n)$ but with a high constant factor, while an order processing service (F_2) scales with $O(n)$ but with a low constant factor, the authentication service becomes the bottleneck, violating the design assumption that processing scales faster than authentication.

To address these challenges, we propose the Ψ framework. It fills the gap by formalizing the concept of *asymptotic order guarantees*. The primary contributions of this paper are:

- The formal definition of Little- ψ for pairwise ordering (Section 3.1).
- The formal definition of Big- Ψ for triple-wise transitive ordering (Section 3.2).
- A set of theorems establishing the properties of these notations, including pairwise implication, transitivity, and quantitative gap bounds (Section 4).
- Comprehensive examples across all major complexity classes (1 , $\log n$, n , $n \log n$, n^2) with valid and invalid cases (Section 5).
- A reference implementation in Python to facilitate adoption (Section 7).

2 Preliminaries

We assume standard asymptotic notation. Let $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ be positive functions. We recall that:

- $f \in O(g)$ iff $\exists c > 0, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- $f \in \Omega(g)$ iff $\exists c > 0, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.
- $f \in \Theta(g)$ iff $f \in O(g)$ and $f \in \Omega(g)$.

3 Proposed Notation

3.1 Little- ψ : Two-Layer Ordering

Definition 3.1 (Little- ψ). Let $F_1, F_2, g : \mathbb{N} \rightarrow \mathbb{R}^+$ be functions. We say the ordered pair (F_1, F_2) is in the class $\psi(g(n))$, denoted $(F_1, F_2) \in \psi(g(n))$, if there exist positive real constants c_1, c_2, c_3 and an integer $n_0 > 0$ such that for all $n \geq n_0$:

$$c_1 \cdot g(n) \leq F_1(n) \leq c_2 \cdot g(n) \leq F_2(n) \leq c_3 \cdot g(n). \quad (1)$$

The constant c_2 plays a pivotal role. It acts as an *asymptotic bridge*:

- The upper bound of F_1 is $c_2g(n)$.
- The lower bound of F_2 is also $c_2g(n)$.

Thus, F_2 is guaranteed to be at least as large as the upper bound of F_1 (up to constants). In practice, this ensures that the downstream stage (F_2) always has "room" to process the output of the upstream stage (F_1) without falling behind asymptotically.

3.2 Big- Ψ : Three-Layer Ordering

Definition 3.2 (Big- Ψ). Let $F_1, F_2, F_3, g : \mathbb{N} \rightarrow \mathbb{R}^+$ be functions. We say the ordered triple (F_1, F_2, F_3) is in the class $\Psi(g(n))$, denoted $(F_1, F_2, F_3) \in \Psi(g(n))$, if there exist positive real constants c_1, C_2, c_3, c_4 and an integer $n_0 > 0$ such that for all $n \geq n_0$:

$$c_1 \cdot g(n) \leq F_1(n) \leq C_2 \cdot g(n) \leq F_2(n) \leq c_3 \cdot g(n) \leq F_3(n) \leq c_4 \cdot g(n). \quad (2)$$

Here, C_2 bridges $F_1 \rightarrow F_2$, and c_3 bridges $F_2 \rightarrow F_3$. The entire chain is now ordered: $F_1 \leq F_2 \leq F_3$ asymptotically.

4 Properties and Theorems

Theorem 4.1 (Pair-wise Implication). If $(F_1, F_2, F_3) \in \Psi(g(n))$, then $(F_1, F_2) \in \psi(g(n))$ and $(F_2, F_3) \in \psi(g(n))$.

Proof. From Equation 2, restrict attention to the left part of the inequality up to F_2 : $c_1 \cdot g(n) \leq F_1(n) \leq C_2 \cdot g(n) \leq F_2(n)$. Since $F_2(n) \leq c_3 \cdot g(n)$ holds from the right part (as c_3 is positive), we have $F_2(n) \leq c_3 \cdot g(n)$. Thus all conditions for $\psi(g(n))$ are satisfied with $c_2 = C_2$. The proof for (F_2, F_3) is symmetrical by taking the suffix. $\square$

Theorem 4.2 (Strong Transitivity). If $(F_1, F_2, F_3) \in \Psi(g(n))$, then there exists an integer n_0 such that for all $n \geq n_0$, $F_1(n) \leq F_2(n) \leq F_3(n)$.

Proof. From Equation 2, we have $F_1(n) \leq C_2g(n) \leq F_2(n)$ and $F_2(n) \leq c_3g(n) \leq F_3(n)$. Hence by transitivity of $\leq$, $F_1(n) \leq F_2(n) \leq F_3(n)$. $\square$

Theorem 4.3 (Quantitative Gap Bounds). If $(F_1, F_2) \in \psi(g(n))$, the asymptotic gap between $F_2(n)$ and $F_1(n)$ is bounded by:

$$(c_2 - c_1)g(n) \leq F_2(n) - F_1(n) \leq (c_3 - c_2)g(n).$$

Proof. $F_2(n) - F_1(n) \geq c_2g(n) - c_1g(n) = (c_2 - c_1)g(n)$. Similarly, $F_2(n) - F_1(n) \leq c_3g(n) - c_2g(n) = (c_3 - c_2)g(n)$. $\square$

This theorem is particularly valuable for capacity planning. It provides a quantitative estimate of the *safety margin* between layers, allowing engineers to provision buffer queues or memory pools.

Corollary 4.3.1. If $(F_1, F_2) \in \psi(g(n))$, then $F_1 \in \Theta(g(n))$ and $F_2 \in \Theta(g(n))$.

Proof. Follows directly from the sandwich inequalities in Equation 1. $\square$

5 Comprehensive Examples

Since both ψ and Ψ require all functions to share the *same* $\Theta(g(n))$ class, we provide valid and invalid configurations across five fundamental complexity families: constant (1), logarithmic ($\log n$), linear (n), linearithmic ($n \log n$), and quadratic (n^2).

5.1 Valid Configurations for Little- ψ

Table 1 lists valid pairs (F_1, F_2) for various $g(n)$.

Table 1: Valid Configurations for $(F_1, F_2) \in \psi(g(n))$				
$g(n)$	$F_1(n)$	$F_2(n)$	Valid Constants (c_1, c_2, c_3)	n_0
1	5	10	(1, 7, 12)	1
$\log n$	$2 \log n + 1$	$3 \log n - 2$	(1, 2.5, 4)	10
n	$2n + 10$	$3n - 5$	(1, 2.5, 4)	20
$n \log n$	$n \log n$	$2n \log n$	(0.5, 1.5, 3)	2
n^2	$n^2 + n$	$2n^2 - n$	(1, 1.5, 3)	2

For the linear case $g(n) = n$:

- Left: $n \leq 2n + 10$ holds for $n \geq 1$.
- Middle 1: $2n + 10 \leq 2.5n \implies n \geq 20$.
- Middle 2: $2.5n \leq 3n - 5 \implies n \geq 10$.
- Right: $3n - 5 \leq 4n$ holds for $n \geq 1$.

Thus $n_0 = 20$ is sufficient.

5.2 Invalid Configurations for Little- ψ

Table 2 shows common pitfalls.

Table 2: Invalid Configurations for $\psi(g(n))$			
$g(n)$	$F_1(n)$	$F_2(n)$	Reason for Invalidity
n	n	n^2	$F_2 \in \Theta(n^2)$ not $\Theta(n)$. No c_2 exists.
n	$\log n$	n	$F_1 \in \Theta(\log n)$ not $\Theta(n)$. Fails lower bound.
n^2	n	n^2	$F_1 \in \Theta(n)$ not $\Theta(n^2)$.
$n \log n$	n	$n \log n$	$F_1 \in \Theta(n)$ not $\Theta(n \log n)$.

5.3 Valid Configurations for Big- Ψ

Table 3 lists valid triples (F_1, F_2, F_3) .

For the linear case: $0.5n \leq n \leq 1.5n \leq 2n \leq 2.5n \leq 3n \leq 4n$ holds for all $n \geq 1$.

Table 3: Valid Configurations for $(F_1, F_2, F_3) \in \Psi(g(n))$

$g(n)$	$F_1(n)$	$F_2(n)$	$F_3(n)$	Valid Constants (c_1, C_2, c_3, c_4)
1	2	5	10	$(1, 3, 7, 12)$
$\log n$	$\log n$	$2 \log n$	$3 \log n$	$(0.5, 1.5, 2.5, 4)$
n	n	$2n$	$3n$	$(0.5, 1.5, 2.5, 4)$
$n \log n$	$n \log n$	$1.5n \log n$	$2n \log n$	$(0.8, 1.2, 1.8, 2.5)$
n^2	n^2	$1.5n^2$	$2n^2$	$(0.8, 1.2, 1.8, 3)$

Table 4: Invalid Configurations for $\Psi(g(n))$

$g(n)$	$F_1(n)$	$F_2(n)$	$F_3(n)$	Reason for Invalidity
n	n	$n \log n$	n^2	Mixed Θ classes. Single g fails.
$\log n$	$\log n$	n	n^2	Mixed classes.
n^2	n	n^2	n^3	F_1 and F_3 not $\Theta(n^2)$.

5.4 Invalid Configurations for Big- Ψ

Table 4 shows triples with mixed complexity classes.

6 Applications

6.1 AI Pipeline Verification

In production AI systems, preprocessing often runs on CPU and inference on GPU. The ratio of CPU to GPU time is critical. By modeling these stages as F_1 and F_2 , engineers can use ψ to mathematically verify that under expected input sizes, the CPU-bound preprocessing will not become a bottleneck. If verification fails, they can increase batch sizes, optimize preprocessing code, or offload more computation to the GPU.

6.2 Compiler Auto-Scheduling

Modern compilers like MLIR [3] utilize multi-level dialects to lower computation graphs. The scheduling of producer and consumer kernels can benefit from Ψ constraints. If a compiler detects that a producer kernel F_1 violates ψ relative to consumer F_2 , it can apply loop tiling, vectorization, or parallelization to the producer to reduce its effective constant factor, thereby restoring the guarantee.

6.3 Service Level Agreements (SLAs)

In microservices, each service has a latency function. If the chain satisfies Ψ , the system can guarantee bounded queue lengths and tail latencies. The gap bounds $(c_2 - c_1)g(n)$ directly translate to buffer sizing requirements.

7 Reference Implementation

We provide a reference implementation in Python, available at <https://github.com/Natarizki/psi-notation>. The core features include:

- `validate_psi(F1, F2, g, n_range)`: Checks the existence of constants via a heuristic grid search over a specified domain.
- `validate_psi_big(F1, F2, F3, g, n_range)`: Extends the check to three layers.
- `plot_bounds(F1, F2, g, constants)`: Visualizes the functions and the bounding constants.
- `auto_fix`: Suggests multiplicative scaling factors to artificially enforce the conditions, aiding in resource allocation simulations.

The implementation includes unit tests and stress tests to handle adversarial inputs and worst-case fluctuations.

8 Related Work

Classical complexity theory is thoroughly covered by Cormen et al. [2] and Sipser [5]. While asymptotic analysis is standard, the explicit study of *order constraints* between multiple functions is less explored. Some works in real-time systems [4] deal with worst-case execution time (WCET) bounds, but they typically focus on absolute bounds rather than relative ordering. Our work complements these by providing a relational perspective.

9 Conclusion and Future Work

We have presented Ψ Notation, a rigorous framework for guaranteeing asymptotic order in two and three-layer systems. We have defined Little- ψ and Big- Ψ , proved their fundamental properties, and demonstrated their practical utility through comprehensive examples covering 1 , $\log n$, n , $n \log n$, and n^2 .

Future work includes:

1. Extending the framework to k -layer systems ($k > 3$).
2. Developing a formal calculus for composing ψ guarantees across complex DAGs (Directed Acyclic Graphs).
3. Integrating the Python validation tool with popular MLIR/LLVM passes to automate ordering enforcement at compile time.
4. Exploring probabilistic variants of ψ for stochastic workloads.

Acknowledgments

The author thanks the open-source mathematics and compiler communities for their invaluable resources and inspiration.

References

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [3] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “MLIR: A Compiler Infrastructure for the End of Moore’s Law,” *arXiv preprint arXiv:2002.11054*, 2020.
- [4] J. W. S. Liu, *Real-Time Systems*. Prentice Hall, 2000.
- [5] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning, 2012.

© 2026 Natarizki.

This work is licensed under the **Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License**.

To view a copy of this license, visit <https://creativecommons.org/licenses/by-nc-nd/4.0/>.